

CHAPTER 5 - ASSEMBLY LANGUAGE BASICS

LOW-LEVEL PROGRAMMING

- **Low-Level Programming** refers to programming languages that are very close to the computer's **hardware** and **instruction set architecture (ISA)**.
- The primary examples are **Machine Language** (binary code) and **Assembly Language**.
- These languages give the programmer **direct control** over hardware resources like CPU registers and memory.

HIGH-LEVEL VS. LOW-LEVEL LANGUAGES

Feature	High-Level Languages (e.g., C++, Python, Java)	Low-Level Languages (e.g., Assembly)
Abstraction	High - Closer to human language; abstract away hardware details.	Low - Closer to the hardware; require detailed hardware knowledge.
Portability	Generally high - Code can run on different machine architectures.	Generally low - Code is specific to a particular ISA (CPU type).
Execution Speed	Usually slower (due to necessary compilation/interpretation).	Can be faster (optimized to the specific hardware).
Development	Faster and easier to write and debug.	Slower , more complex, and error-prone.
Memory Management	Automatic or simplified (handled by the compiler/runtime).	Manual (programmer manages memory directly).
Use Case	Applications, web development, general programming.	Operating systems, device drivers, performance-critical code, embedded systems.

BASICS OF ASSEMBLY LANGUAGE

- **Assembly Language** is a **symbolic representation** of the computer's **machine code**.
- **Mnemonics**: Short, easy-to-remember symbolic names for machine operations. They are human-readable representations of the operation codes (opcodes).
 - *Examples*: **MOV** (move data), **ADD** (addition), **SUB** (subtraction), **JMP** (jump/branch).
- **Opcodes (Operation Codes)**: The **binary bit patterns** that represent the basic operations the CPU can perform (e.g., the machine code for ADD). Mnemonics are a direct mapping to opcodes.

- **Operands:** Specify the data or locations (like registers or memory addresses) on which the operation will be performed.

STRUCTURE OF AN ASSEMBLY PROGRAM

A typical assembly program often has sections (or *segments*), though the specifics vary by assembler and architecture:

1. **Data Segment:** Used to define and initialize **variables** and **constants** (data that the program will operate on).
 2. **Code Segment (Text Segment):** Contains the **actual instructions** (mnemonics and operands) that the CPU will execute. This is where the program logic resides.
 3. **Stack Segment:** Used for temporary data storage, function parameters, local variables, and return addresses for procedure calls.
- *Key Components:* **Labels** (names for memory locations or instructions), **Directives** (commands for the assembler, not the CPU), **Instructions** (the actual code).

ASSEMBLER: HOW IT CONVERTS CODE INTO MACHINE INSTRUCTIONS

- An **Assembler** is a program that translates an **Assembly Language program** into **Machine Language** (binary code) that the computer's CPU can directly execute.
- **Two-Pass Process (Common Method):**
 1. **Pass 1:** Scans the entire source code to build a **Symbol Table**. The Symbol Table maps all symbolic labels and variable names to their corresponding **memory addresses**. No machine code is generated yet.
 2. **Pass 2:** Uses the Symbol Table to translate each mnemonic and its operands into the **corresponding binary opcode and operand addresses/values**. This generates the final **object file** (machine code).
- **Linker:** Often used after the assembler to combine one or more object files into a single, executable program, resolving references between modules.

Example Program: Addition of Two Numbers (Conceptual/Illustrative)

- *Goal:* Calculate $A + B$ and store the result in C .
- *Architecture:* Assumed to be a simple RISC-like architecture.

Instruction (Assembly)	Comment
MOV R1, A	Move the value stored in memory location A into Register 1 (R1) .
MOV R2, B	Move the value stored in memory location B into Register 2 (R2) .

ADD R3, R1, R2	Add the contents of R1 and R2 , store the result in Register 3 (R3) .
MOV C, R3	Move the result from R3 back to the memory location C .

ASSEMBLY INSTRUCTION FORMAT

While the exact format varies widely by ISA (e.g., x86, ARM, MIPS), a general format is:

LABEL (optional) MNEMONIC OPERANDS COMMENT (optional)

- **LABEL:** A symbolic name for the memory address of the instruction.
- **MNEMONIC:** The operation (e.g., MOV, ADD). This maps directly to the **opcode**.
- **OPERANDS:** Specify the source data, destination location (register or memory), or immediate values. The number and type depend on the instruction and ISA.
- **COMMENT:** Ignored by the assembler, used by the programmer for clarity.

LIMITATIONS OF ASSEMBLY

1. **Machine Dependence (Low Portability):** Code is tied to the **specific processor architecture** (ISA). Code for an x86 CPU won't run on an ARM CPU without re-assembly and significant modifications.
2. **High Development Cost:** Programs are **long, complex, and time-consuming** to write and debug. A simple operation in a high-level language can take multiple assembly instructions.
3. **Low Readability and Maintainability:** The low level of abstraction makes assembly code **hard to read, understand, and maintain**, especially for large programs.
4. **Steep Learning Curve:** Requires **in-depth knowledge** of the specific processor's registers, memory structure, and instruction set.